



AWS CodePipeline

Introduction

AWS CodePipeline is a **fully managed CI/CD service** that automates the **build, test, and deployment process** for applications.

- What is AWS CodePipeline?
- Why is it important in DevOps workflows?
- How can developers configure AWS CodePipeline for automation?

This guide explains **how AWS CodePipeline enables continuous delivery of software.**

Section 1: Learn

What is AWS CodePipeline?

AWS CodePipeline automates **software release processes**, ensuring applications **move smoothly from development to production.**

Why Use AWS CodePipeline?

Feature	Benefit
End-to-End Automation	Orchestrates the entire CI/CD pipeline
Integration with AWS Services	Works with CodeCommit, CodeBuild, and CodeDeploy
Scalability	Handles deployments across multiple regions
Customizable Pipelines	Supports third-party tools like Jenkins, GitHub



Feature	Benefit
Security & IAM Control	Manages user access via AWS IAM

AWS CodePipeline Workflow

Stage	AWS Service	Purpose
Source Control	AWS CodeCommit or GitHub	Stores the source code
Build	AWS CodeBuild	Compiles and tests the application
Test	AWS CodeBuild or third-party tools	Runs automated tests
Deploy	AWS CodeDeploy or Lambda	Deploys the application
Approval Process	Manual/Automated Approval	Ensures verification before release

Interesting Fact

AWS CodePipeline **releases software updates multiple times per day**, reducing **deployment failures by up to 90%**.



Section 2: Practice

1. Setting Up a CI/CD Pipeline in AWS CodePipeline

Step 1: Create a CodePipeline

1. Open **AWS CodePipeline Console** → Click **Create Pipeline**.
2. Enter **Pipeline Name** → Choose **New Service Role**.
3. Select **Source Provider** (AWS CodeCommit, GitHub, Bitbucket).

Step 2: Configure Build Stage

1. Select **Build Provider: AWS CodeBuild**.
2. Choose an **existing or new CodeBuild project**.
3. Define a **buildspec.yml** file in your repo:

```
version: 0.2

phases:

  install:

    runtime-versions:

      java: corretto11

  build:

    commands:

      - mvn clean install

      - mvn test
```

Step 3: Deploy with AWS CodeDeploy

1. Select **Deploy Provider: AWS CodeDeploy**.
2. Choose an **existing CodeDeploy application**.
3. Click **Create Pipeline**.



Now, every **code push** triggers an automated deployment.

Why is this useful?

- Reduces **manual deployment efforts and errors**.
-

2. Adding Manual Approval Before Deployment

AWS CodePipeline allows manual approval before production deployment.

Modify pipeline settings to include an **Approval Stage**:

stages:

- name: Deploy

actions:

- name: Approve_Deployment

actionTypeId:

category: Approval

provider: Manual

Now, AWS CodePipeline **pauses for manual approval** before deployment.

Why is this useful?

- Ensures **human verification before critical releases**.
-

Section 3: Know More

Frequently Asked Questions

1. **What is AWS CodePipeline used for?**
 - Automating **build, test, and deployment** workflows.
2. **Can AWS CodePipeline integrate with Jenkins?**
 - Yes, it can trigger **Jenkins jobs** as part of the pipeline.



3. **How do I monitor AWS CodePipeline?**
 - Use **AWS CloudWatch Logs** and **AWS Console Dashboard**.
 4. **What happens if a CodePipeline stage fails?**
 - **AWS stops execution and logs the error** for debugging.
 5. **Is AWS CodePipeline free?**
 - No, pricing depends on **pipeline execution and storage**.
-

Conclusion

AWS CodePipeline **streamlines software delivery** by automating **CI/CD workflows**.

Start by **creating a pipeline**, **integrating AWS CodeBuild**, and **deploying with AWS CodeDeploy**, and soon you'll be **automating software releases like a pro!**